STARGAZE MPP

Backend Infrastructure

Express 5 | tRPC 11 | elizaOS v2 | PostgreSQL | TimescaleDB | Redis | Tempo SDK | Solana MPP SDK | ZK Proof Pipeline | Reputation Oracle

StargazeIndex Service | MPP Session Manager | StargazeVault ZK Pipeline | Reputation Oracle | Payment Router | Provider Registry | No KYC/KYB

v1.0.0 | May 2026 | Confidential & Proprietary

1. Architecture Overview

StargazeMPP backend has six core services. StargazeIndex Service: provider registry, discovery, category management. MPP Session Manager: session lifecycle, voucher validation, batch settlement. StargazeVault Pipeline: ZK proof generation and verification for private intelligence. Reputation Oracle (elizaOS v2): crowd-vote aggregation, accuracy scoring, slash execution. Payment Router: PathUSD and USDC routing, fee burns, staker distributions. Solana Indexer (Rust + Yellowstone): all on-chain events at sub-50ms lag. No PII stored. Wallet address is complete identity.

Service	Technology	Role
API Gateway	Express 5 + tRPC 11	All endpoints -- no PII stored or accepted
StargazeIndex Service	Node 20+ + PostgreSQL	Provider registry, discovery, category management
MPP Session Manager	Node 20+ + Tempo SDK + solana-mpp	Session lifecycle, voucher validation, batch settlement
Vault ZK Pipeline	Node 20+ + snarkjs + Groth16	ZK proof generation/verification for VAULT providers
Reputation Oracle	elizaOS v2 + Claude API	Crowd-vote aggregation, accuracy scoring, slash execution
Payment Router	Node 20+ + Tempo SDK	PathUSD/USDC routing, $GAZE fee burn, staker distribution
Solana Indexer	Rust + Yellowstone gRPC	All Solana-side MPP and on-chain events < 50ms
Database	PostgreSQL + TimescaleDB	Providers, sessions, queries, reputation, earnings
Cache	Redis (Upstash)	Session state, provider lookup, rate limits, voucher queue
ORM	Drizzle ORM	Type-safe schema + migrations
LLM	Claude API (Anthropic)	Reputation Oracle accuracy assessment
Runtime	Bun / Node 20+	TypeScript native

2. Smart Contracts / On-Chain Programs

Contract / Program	Specification
StargazeRegistry (Tempo EVM)	Provider registration. $GAZE stake collection and slashing. Reputation score storage. Verified Provider badge issua
StargazeEscrow (Tempo EVM)	MPP session escrow deposits. Voucher batch settlement -- receives signed settlement from Session Manager. Unus
GAZEToken (Tempo EVM)	ERC-20 $GAZE token. Transfer hook -> BurnController on routing events. Staking: 7-day unstake cooldown. Weekl
BurnController (Tempo EVM)	Executes $GAZE burns atomically: routing fee burns (50% of 2%), citation burns (5 GAZE), reputation vote burns (1

PrivacyVaultRegistry (Tempo EVM)	ZK verifier contract addresses per provider. Buyer-key rotation config. Auditor key management. Confidential payme
StargazeAnchor (Solana)	Solana-side StargazeRegistry mirror for Solana-native providers. Bridges to Tempo via Chainlink CCIP for unified re

```
# Deployment order -- strict
1. GAZEToken -> ERC-20 with staking and governance
2. BurnController -> depends GAZEToken
3. StargazeEscrow -> holds PathUSD session deposits
4. StargazeRegistry -> depends GAZEToken + BurnController
5. PrivacyVaultRegistry -> depends StargazeRegistry
6. StargazeAnchor (Solana) -> CCIP bridge to Tempo
# ALL: 4-of-7 Safe multisig upgrade authority from day one
# ALL: Trail of Bits audit before mainnet
# GAZEToken: verified ERC-20 with Certora formal verification on transfer hook
```

3. MPP Session Manager -- The Core Service

The MPP Session Manager handles the complete session lifecycle: escrow deposit, voucher validation, settlement batching, and refund. It supports both Tempo PathUSD sessions and Solana USDC sessions (via x402). Voucher verification via ecrecover -- no RPC, no database per validation. Target: sub-10ms voucher verification latency. Sessions persist in Redis with 24h TTL. Settlement runs on session close OR when session balance falls below 10% of deposit.

```
// MPP Session Manager -- core session lifecycle
// 1. OPEN SESSION
// Agent sends deposit tx on Tempo (PathUSD) or Solana (USDC)
// Session Manager validates tx, creates session state in Redis
// Returns session_token (JWT: agentWallet + sessionId + spendingLimit + expiry)
async function openSession(deposit: DepositProof): Promise {
await verifyDepositOnChain(deposit); // Tempo or Solana RPC
const session = { id: uuid(), agentWallet, balance: deposit.amount, queries: 0 };
await redis.set(`session:${session.id}`, session, { ex: 86400 });
return signSessionToken(session);
}
// 2. VALIDATE VOUCHER (sub-10ms -- no RPC, no DB)
// Agent sends cumulative EIP-712 signed voucher with each request
// ecrecover recovers agent wallet from signature -- no lookup needed
// Cumulative amount compared to session balance in Redis
function validateVoucher(voucher: EIP712Voucher, sessionId: string): boolean {
const agentWallet = ecrecover(voucher.signature, voucher.hash);
const session = redis.get(`session:${sessionId}`); // sub-ms Redis read
return session.agentWallet === agentWallet
&& voucher.cumulativeAmount <= session.spendingLimit
&& voucher.cumulativeAmount > session.lastVoucherAmount;
}
```

```
// 3. SETTLE (batch -- called on session close or balance threshold)
// Session Manager submits single on-chain tx covering all vouchers
// StargazeEscrow releases: 98% to providers, 2% to routing fee
// BurnController burns 50% of routing fee in $GAZE
// Remaining 50% routing fee to GAZEToken staker pool
// Unused balance refunded to agent wallet in same tx
async function settleSession(sessionId: string): Promise {
const session = await redis.get(`session:${sessionId}`);
const vouchers = session.voucherBatch; // all vouchers accumulated
const tx = await stargazeEscrow.settle(sessionId, vouchers);
await redis.del(`session:${sessionId}`);
return tx;
}
```

4. StargazeVault ZK Pipeline -- Privacy Intelligence

The StargazeVault pipeline enables intelligence providers to serve ZK-proven aggregate answers without ever revealing underlying data. Built with snarkjs + Groth16. Compatible with the Axonmed and Synapse ZK infrastructure already in the ecosystem. Provider submits their ZK verifier spec at registration. Pipeline generates proofs on each query. Agent receives: public output + Groth16 proof hash. Proof verifiable by anyone. Underlying data never leaves provider.

```
// StargazeVault ZK proof generation pipeline
// Provider registration: submits ZK circuit spec
// { circuitWasm, provingKey, verifyingKey, publicInputSchema, privateInputSchema }
// On agent query to a VAULT provider:
// 1. StargazeVault receives: query body (what agent wants to know)
// 2. Provider endpoint fetches private data (health records, telemetry, research)
// 3. ZK circuit computes: public_output = f(private_data, query_params)
// 4. Groth16 proof generated: proves computation is correct without revealing private_data
// 5. Agent receives: { publicOutput, proofHash, receipt }
// 6. Agent (or anyone) can verify: snarkjs.verify(verifyingKey, publicOutput, proof)
import { groth16 } from 'snarkjs';
async function generateVaultProof(query: VaultQuery, provider: VaultProvider) {
// Fetch private inputs from provider's secure endpoint
const privateInputs = await provider.fetchPrivateData(query);
// Compute ZK proof
const { proof, publicSignals } = await groth16.fullProve(
{ ...privateInputs, ...query.publicParams },
provider.circuit.wasm,
provider.circuit.provingKey
);
// Verify proof is valid before returning to agent
const valid = await groth16.verify(
```

```
provider.circuit.verifyingKey,
publicSignals,
proof
);
if (!valid) throw new Error('ZK proof invalid -- computation error');
return {
publicOutput: publicSignals, // what agent receives
proofHash: hashProof(proof), // verifiable by anyone
// proof itself available via Arweave CID for full verification
};
}
```

5. Reputation Oracle -- elizaOS v2 Agent

The Reputation Oracle is an elizaOS v2 agent that runs continuously, monitoring provider quality across three dimensions: automated latency and uptime monitoring, crowd-sourced accuracy votes from querying agents, and AI-assisted response quality assessment via Claude API. Oracle outputs are signed and committed on-chain. Providers with score below 600 are flagged. Below 400 triggers review. Proven fraud: oracle proposes slash to DAO, executed within 48h.

Function	Tools	How it works
Latency monitor	elizaOS + synthetic probe agent	Runs 5 test queries per provider per hour. Tracks p50/p95/p99 latency. Updates upti
Accuracy aggregator	elizaOS + on-chain vote reader	Reads crowd-verification votes from StargazeRegistry. Agents burn 1 $GAZE per vo
AI quality assessor	elizaOS + Claude API	For providers with complaint volume above threshold: Claude assesses sample resp
Slash proposer	elizaOS + StargazeRegistry CPI	When fraud is confirmed: proposes slash to DAO governance. DAO votes within 48h

6. Database Schema (Key Tables)

providers

Column	Type	Notes
id	text PK	provider_timestamp_random
wallet_address	text NOT NULL	Tempo/Solana wallet -- pseudonymous identity
name	text NOT NULL	Display name in StargazeIndex
category	text NOT NULL	on-chain-analytics|physical-ai|desci|rwa|compliance|ai-model
price_charge	numeric(12,6)	PathUSD per one-shot query

price_session	numeric(12,6)	PathUSD per session voucher
privacy_tier	text NOT NULL	OPEN|ZK-VAULT|CONFIDENTIAL|BUYER-KEY
gaze_staked	numeric(20,6)	Staked $GAZE (slashable)
reputation_score	integer	0-1000 composite score
verified	boolean	True when score >800 and stake >=500 GAZE
registered_at	timestamp	TimescaleDB partition key

sessions

Column	Type	Notes
id	text PK	session_timestamp_random
agent_wallet	text NOT NULL	Querying agent wallet (pseudonymous)
deposit_amount	numeric(20,6)	PathUSD deposited in StargazeEscrow
method	text NOT NULL	tempo|solana|stripe|visa|lightning
spending_limit	numeric(20,6)	Agent-defined maximum spend
current_spend	numeric(20,6)	Running total of settled vouchers
query_count	integer	Number of queries in this session
status	text NOT NULL	open|settled|expired
settlement_tx	text NULL	On-chain settlement transaction hash
opened_at	timestamp	TimescaleDB partition key

queries

Column	Type	Notes
id	uuid PK	Auto UUID
session_id	text FK	References sessions.id
provider_id	text FK	References providers.id
voucher_amount	numeric(12,6)	PathUSD for this specific query
latency_ms	integer	End-to-end response latency
vault_proof_hash	text NULL	ZK proof hash if VAULT provider
accuracy_votes	integer	Net accuracy votes (positive = accurate)
executed_at	timestamp	TimescaleDB partition key

```
SELECT create_hypertable('providers', 'registered_at');
SELECT create_hypertable('sessions', 'opened_at');
SELECT create_hypertable('queries', 'executed_at');
```

7. tRPC Routes (/trpc)

index

Procedure	Type	Auth	Description
index.getProviders	query	Public	All providers with filters: category, price, privacy, reputation, method
index.getProvider	query	Public	Single provider full detail: pricing, privacy spec, sample response
index.search	query	Public	Full-text search across name, category, tags
index.getLiveStats	query	Public	Active providers, 24h queries, PathUSD settled, $GAZE burned

session

Procedure	Type	Auth	Description
session.open	mutation	Wallet	Open MPP session: validate deposit, create session, return token
session.query	mutation	Token	Route query to provider: validate voucher, proxy request, return result
session.close	mutation	Token	Close session: trigger batch settlement, return refund amount
session.getStatus	query	Token	Current session: balance, spend, query count, provider breakdown

provider

Procedure	Type	Auth	Description
provider.register	mutation	Wallet	Register MPP service: config, pricing, privacy, $GAZE stake
provider.update	mutation	Wallet	Update pricing, privacy config, accepted methods
provider.getDashboard	query	Wallet	Earnings, query analytics, reputation, session volume
provider.withdraw	mutation	Wallet	Withdraw earned PathUSD to provider wallet

vault

Procedure	Type	Auth	Description
vault.configure	mutation	Wallet	Set ZK verifier spec, privacy tier, auditor key for VAULT providers
vault.testProof	mutation	Wallet	Run sample ZK proof -- validates circuit before going live
vault.getSpec	query	Public	Public ZK verifier spec for any VAULT provider

reputation

Procedure	Type	Auth	Description
reputation.vote	mutation	Wallet	Cast accuracy vote (ACCURATE/INACCURATE) -- burns 1 $GAZE
reputation.getHistory	query	Public	Full reputation history for any provider
reputation.getScore	query	Public	Current score breakdown: accuracy, uptime, latency, refund

gaze

Procedure	Type	Auth	Description
gaze.getStats	query	Public	Total staked, burn rate, routing fee revenue, staker APY
gaze.getBurnFeed	query	Public	Live $GAZE burn events from routing, citations, votes
gaze.stake	mutation	Wallet	Stake $GAZE to GAZEToken staking contract
gaze.unstake	mutation	Wallet	Unstake $GAZE -- 7-day cooldown starts
gaze.claimRewards	mutation	Wallet	Claim PathUSD/USDC staking rewards

8. Provider SDK -- @stargazempp/provider-sdk

Any HTTP endpoint becomes a monetized MPP intelligence service with one decorator. The SDK handles the full 402 challenge/credential flow, voucher validation via ecrecover, session tracking, and receipt generation. npm install @stargazempp/provider-sdk

```
import { StargazeProvider } from '@stargazempp/provider-sdk';
const provider = new StargazeProvider({
serviceId: 'my-intelligence',
category: 'on-chain-analytics',
pricing: { currency: 'PathUSD', chargeIntent: 0.008, sessionIntent: 0.004 },
privacy: 'zk-aggregate', // OPEN | zk-aggregate | confidential | buyer-key
methods: ['tempo', 'solana', 'stripe', 'visa'],
gazeStake: 50,
apiKey: process.env.STARGAZE_API_KEY,
});
// Wrap any existing Express/Hono/Fastify route
app.post('/api/intelligence', provider.monetize(async (req, payment) => {
// payment.voucher: validated by SDK via ecrecover (no RPC)
// payment.amount: confirmed against provider pricing
// payment.method: tempo | solana | stripe | visa
const result = await yourLogic(req.body);
return { data: result, receipt: payment.receipt };
}));
// For StargazeVault ZK-AGGREGATE providers:
```

```
app.post('/api/private-intelligence', provider.vaultMonetize(
{ circuit: './circuit.wasm', provingKey: './proving_key.zkey' },
async (req, payment) => {
const privateData = await fetchSensitiveData(req.body.cohortId);
// SDK generates ZK proof automatically
// Agent receives publicOutput + proofHash only
return { privateInputs: privateData, publicParams: req.body };
}
));
```

9. Environment Variables + Deployment

```
PORT=3001
DATABASE_URL=postgres://stargaze:pw@localhost:5432/stargaze
REDIS_URL=redis://localhost:6379
TEMPO_RPC=https://rpc.tempo.xyz
TEMPO_PRIVATE_KEY=your_tempo_wallet_key # payment router + settlement
HELIUS_API_KEY=your_helius_key # Solana MPP + indexer
ANTHROPIC_API_KEY=your_anthropic_key # Reputation Oracle Claude agent
STARGAZE_ESCROW_ADDRESS=0x... # StargazeEscrow on Tempo EVM
STARGAZE_REGISTRY_ADDRESS=0x... # StargazeRegistry on Tempo EVM
GAZE_TOKEN_ADDRESS=0x... # GAZEToken on Tempo EVM
SOLANA_ANCHOR_PROGRAM=StargazeXxx... # StargazeAnchor on Solana
CCIP_ROUTER_TEMPO=0x... # Chainlink CCIP for cross-chain
ARWEAVE_KEY=your_arweave_key # ZK proof permanent storage
CORS_ORIGIN=https://app.stargazempp.xyz
```

```
git clone https://github.com/stargazempp/stargaze && cd stargaze
bun install
cd docker && docker compose up -d # PostgreSQL + TimescaleDB + Redis
cd packages/backend && cp .env.example .env # fill Tempo RPC + Anthropic
bun run db:push
psql $DATABASE_URL -c "SELECT create_hypertable('providers','registered_at');"
psql $DATABASE_URL -c "SELECT create_hypertable('sessions','opened_at');"
psql $DATABASE_URL -c "SELECT create_hypertable('queries','executed_at');"
bun run dev # -> http://localhost:3001 | health: /health
```

Service	Provider	Notes
API + Reputation Oracle	Railway / Fly.io	elizaOS Reputation Oracle as persistent background process
Frontend	Vercel	Next.js 15 -- deep space / star violet UI
PostgreSQL	Neon / Supabase	TimescaleDB for session + query time-series required
Redis	Upstash	Session state, voucher queue, provider cache, rate limits
Tempo RPC	Tempo official RPC	Escrow settlement, $GAZE burns, registry reads

Solana RPC	Helius dedicated	Solana-side MPP payments + Anchor program events
Arweave	Irys (Bundlr)	ZK proof permanent storage -- verifiable by anyone forever
CCIP bridge	Chainlink official	$GAZE and provider registry cross-chain Tempo <> Solana
Alerting	PagerDuty	Reputation Oracle anomalies, settlement failures, provider slashes

10. Security Architecture

Control	Implementation
StargazeEscrow reentrancy	ReentrancyGuard + nonReentrant on all state-changing settlement functions
Upgrade authority	4-of-7 Safe multisig. 14-day timelock on all Tempo EVM contract changes
Voucher replay prevention	Cumulative EIP-712 vouchers -- amount must strictly increase per session
ZK proof soundness	snarkjs Groth16 -- cryptographic soundness, not just collision resistance
Provider fraud	Reputation Oracle detects schema violations. DAO slash within 48h
Session hijack	Session token JWT bound to agent wallet -- ecrecover validates every voucher
Payment privacy	Tempo confidential tx for CONFIDENTIAL tier -- auditor key for compliance
Bug bounty	Immunefi: Critical $200K (escrow drain). High $50K. Live at launch.
Audit	Trail of Bits (contracts). Groth16 circuit review pre-VAULT launch.